Flutter 앱 개발 · 신입

권채린

연락처

이름	권채린
이메일	kchaerin0811@gmail.com
GitHub	github.com/CR-KStar/diet_project
트러블슈팅	cr-kstar.github.io/diet_project/troubleshooting/
앱 · 웹	App Store · 웹 앱 · 웹 체험판

식단 · 운동 · 체중 기록과 식물 키우기를 한 앱에서

식단 기록 앱 개발

밋그린 (MEATGREEN)

식사 사진을 찍으면 AI가 칼로리와 영양소를 추정하고, 기록할수록 식물이 자라요. 기획부터 구현, 서버, **App Store · Google Play 제출과 심사 대응**까지 한 사이클을 만들었어요.

사진 한 장으로 간편 기록	목표 대비 한눈에 확인	식물 성장으로 동기 부여	친구와 함께 꾸준히 유지

- App Store 출시 (2026.10.16)
- Google Play 출시 심사 중
- 웹 앱 배포

권채린 · Flutter 앱 개발 · 신입 · kchaerin0811@gmail.com

기술 스택

앱	Flutter / Dart · Provider · ViewModel · Repository
서버	Firebase Auth · Firestore · Cloud Functions (TS) · Firebase Hosting
AI · 인증	Claude API · Sign in with Apple / Google
도구	Git / GitHub · GitHub Pages · Claude Code

주요 특징 (구현한 기능)

식사 사진 AI 분석
앱에서 Claude를 바로 부르면 키가 노출돼서 Cloud Functions를 한 번 거치게 했어요. 서버가 로그인한 사람인지, 오늘 30번을 안 넘었는지 확인하고, 결과는 정해진 JSON 형태로만 받아요. 사진은 내 폰에만 보관해요

하루 목표 관리
키 · 몸무게 같은 신체 정보로 기초대사량을 계산해 하루 권장 칼로리를 정하고, 영양소마다 얼마나 채웠는지 바로 보여 줘요

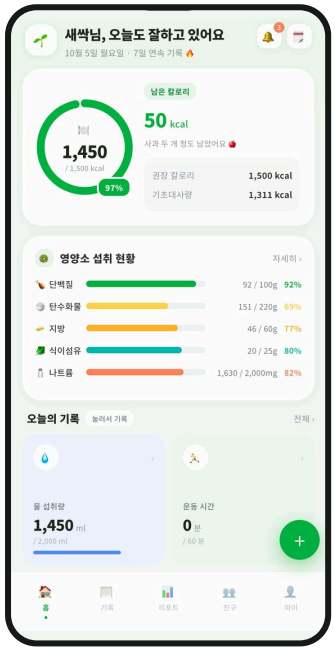
기록 & 리포트
기록 하나가 Firestore 문서 하나예요. 날짜를 '2026-10-18' 같은 문자열로 저장해서 한 달치도 한 번에 불러와요. 고치거나 지울 때는 서버에 먼저 반영하고, 실패하면 화면도 원래대로 돌려놔요

식물 성장 보상
물 · 운동 · 식단을 기록하면 EXP가 쌓이고, 이틀 넘게 쉬면 식물이 시들어요. 다시 기록하면 살아나요. 며칠 쉬었는지를 날짜로 계산하는 규칙을 직접 만들었어요

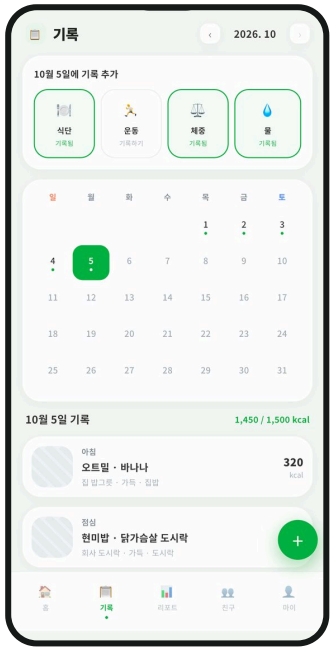
친구 & 챌린지
초대 코드를 입력하면 상대를 찾아 친구 요청을 보내요. 요청은 보낸 사람만 만들고 수락은 받은 사람만 할 수 있게 Firestore 보안 규칙으로 막았어요. 친구 삭제 · 차단 · 신고도 돼요

핵심 화면 (실제 앱 화면)

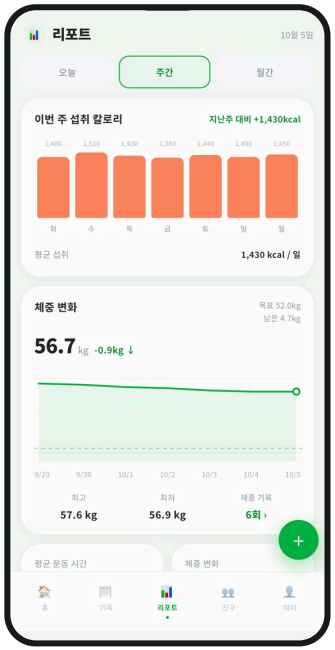
홈 (오늘)



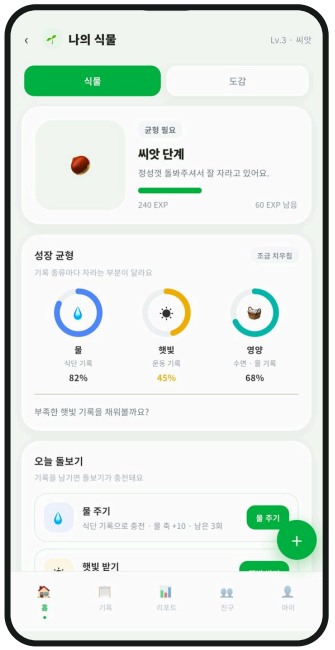
식단 기록



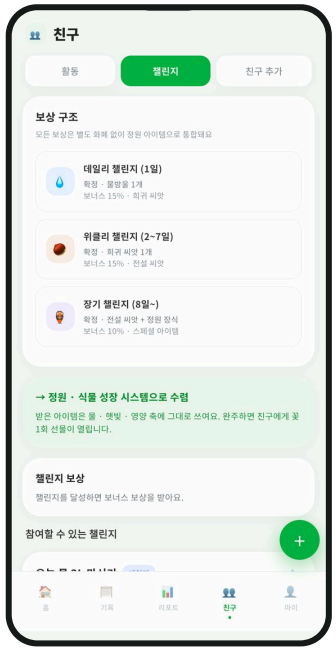
리포트 (주간)



식물 성장



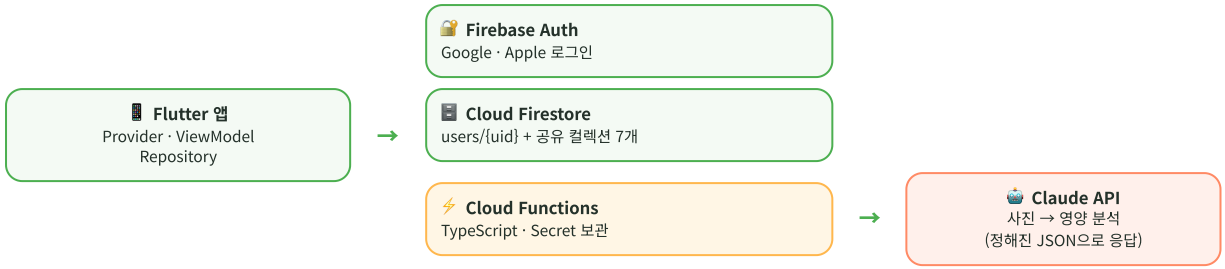
친구 · 챌린지



서버 구조와 문제 해결

현재 상태 (2026년 10월): iOS 출시 · 웹 앱 배포 · Android 출시 심사 중

서버 구조



Firestore 보안 규칙 — 컬렉션마다 최소 권한 · 웹 앱 · 웹 체험판은 Firebase Hosting으로 배포

공유 컬렉션: publicProfiles · inviteCodes · friendRequests · friendships · challenges · challengeProgress · activityFeed

출시 현황

- **iOS** — **App Store 출시 (2026.10.16)**
  심사 반려 2건을 원인 분석 · 수정 후 재제출해 승인
- **Android** — **Google Play 출시 심사 중**
  비공개 테스트를 마치고 프로덕션 출시를 신청
- **Web** — **웹 앱 배포 (meatgreen.web.app) · 웹 체험판**
  Firebase Hosting — Google 로그인으로 쓰는 웹 앱과 로그인 없는 체험판을 따로 배포

만든 방식 · 앱 아키텍처

UI	화면 위젯 (홈 · 기록 · 리포트 · 식물 · 친구) · 디자인 토큰
▼ 상태 구독 (Provider)	
상태	ViewModel 21개(식단 · 운동 · 식물 · 친구 · 화면 이동 …) — 화면이 각자 구독 + AppState(여러 기능 연결)
▼ 인터페이스 호출	
Repository	저장소 인터페이스 → **Firestore 구현 / 메모리 구현** (테스트 · 웹 데모)
▼	
서비스	인증 · AI 분석 · 알림 · 앱 아이콘 (Firebase, Cloud Functions)
모델	domain/models — 모든 모델이 id · userId를 가지고, 코덱이 저장 형식으로 변환

- ✔ **인터페이스 뒤에 저장소를 숨겨서** 서버 없이 테스트하고 웹 체험판을 만들 수 있어요
- ✔ 화면은 상태만 구독하고, **저장 · 서버 접근은 Repository 뒤**로 모았어요
- ✔ 3,400줄 상태 클래스를 영역별 **ViewModel 21개로 분리**했어요 (AppState는 약 1,300줄 · 여러 기능을 엮는 연결만)

Claude Code와 협업해 개발했고, 기획 방향과 검증, **아키텍처 구조 분리(ViewModel·Repository)**, **DB·백엔드 설계(Firestore 데이터 모델링 · 보안 규칙 · Cloud Functions · 인증과 계정 삭제 흐름)**, 스토어 배포, 심사 대응은 직접 진행했어요. DB·백엔드는 지금 처음부터 다시 구현해 보며 더 깊이 공부하고 있어요. 앱 아이콘 캐릭터는 AI로 생성했어요.

01 스토어 제출과 심사 대응

- iOS 서명 · TestFlight · App Privacy, Play 데이터 보안 · 개인 계정 **비공개 테스트**(테스터 12명 · 14일)
- 반려 **2건**(건강 정보 출처 1.4.1, Apple 로그인 2.1)을 원인 분석 후 수정, 재제출해 **승인·출시**

02 Firestore 모델링과 보안 규칙

- 내 기록은 **users/{uid} 하위**, 함께 쓰는 데이터는 최상위로 분리
- 컬렉션별 규칙(예: 친구 요청은 보낸 사람만 생성 · 받은 사람만 수락), 날짜를 **정렬 가능한 키**로 저장
- 한도 대응: 삭제 300개씩, `whereIn` 30개씩

03 서버리스 AI 파이프라인

- 사진을 **서버 함수**로 보내 Claude가 음식 · 칼로리 · 영양소를 **정해진 JSON**으로 반환
- API 키는 서버 Secret에만 보관, 한국어 이름 지시를 프롬프트와 스키마 양쪽에 반영
- 외부 전송 전 **사용자 동의** 팝업

04 계정 수명주기와 데이터 삭제

- Apple 로그인 **nonce(SHA-256)**, 삭제 순서: 재인증 → 토큰 철회 → 데이터 삭제 → 계정 삭제
- 빠뜨린 소셜 데이터를 찾아 저장소 · **보안 규칙** · 처방침 문구를 함께 수정

05 Repository 추상화와 구조 분리

- 저장소 인터페이스에 **Firestore · 메모리** 두 구현 → 서버 없이 웹 데모 · 테스트
- 3,400줄 상태 클래스를 영역별 **ViewModel**로 분리 완료

06 사용자 경험의 세부 완성도

- 기록 여부에 따라 바뀌는 **동적 앱 아이콘**(iOS 대체 아이콘 · Android activity-alias)
- 글자 크기 배율 · 디자인 토큰 · 화면 오버플로 점검, 로그인 없는 **웹 체험판**

개발 · 배포 흐름

설계·구현 — Flutter + Firebase
검증 — 자동 테스트로 점검
내부 배포 — TestFlight · Play 내부
App Store 출시 — 반려 2건 수정 후 승인
Play — 출시 심사 중

백엔드 · DB 설계

Firebase — Auth · Firestore · Cloud Functions

Firestore 데이터 구조

```
users/{uid}                    프로필 (닉네임 · 신체 정보 · 알림 · 공개 범위)
 ├ weight · exercise · meals   기록 — 문서 1건 = 기록 1건
 │  bowls · routines · water
 ├ plant/state                 내 식물 (경험치 · 축 점수 · 누적 기록)
 └ dexEntries/{speciesId}      식물 도감
publicProfiles/{uid}           친구 찾기용 닉네임 · 초대 코드
inviteCodes/{code}             초대 코드 → uid
friendRequests · friendships   친구 요청 · 확정된 관계
challenges · challengeProgress 챌린지 · 참가자별 진행
activityFeed                   친구 활동 한 줄
```

- ✔ 내 기록은 **users/{uid} 아래** → 규칙 한 줄로 하위 전체를 보호
- ✔ 사용자 **사이의 관계**(친구 · 챌린지)는 한 사람 아래에 둘 수 없어서 최상위로 분리
- ✔ 날짜는 **'yyyy-MM-dd' 문자열**(dateKey) → 사전순이 곧 시간순이라 월 범위 조회가 **색인 없이** 한 필드 조건으로 됨
- ✔ 모델 ↔ 저장 형식 변환은 **codec**이 맡고, 예전 형식 · 깨진 값도 읽도록 처리

보안 규칙 — 기본은 전부 막고, 필요한 것만 열기

컬렉션	읽기	쓰기
users/{uid}/**	본인	본인
publicProfiles · inviteCodes	로그인한 사용자 (코드로 찾아야 해서)	본인 것만
friendRequests	보낸 사람 · 받은 사람	생성은 **보낸 사람**, 수락 · 거절은 **받은 사람**
friendships	관계에 포함된 두 사람	포함된 사람
challenges	로그인한 사용자	만든 사람(creatorId)이 생성 · 삭제, 참여는 로그인한 사용자
challengeProgress · activityFeed	로그인한 사용자	본인(uid) 것만, 삭제도 본인 것만

규칙은 **request.auth.uid**(요청한 사람) · **resource.data**(저장된 값) · **request.resource.data**(쓰려는 값)로 조건을 써요. 앱이 DB에 직접 접근하는 구조라 **규칙이 곧 서버의 권한 검사**예요.

A 사진 분석 — 앱 → 함수 → Claude

1. 동의 팝업 후, 사진(base64)과 그릇 정보를 **Cloud Function**(analyzeMealPhoto)으로 보내요
2. 함수가 **서버 Secret의 API 키**로 Claude를 호출하고, **정해진 JSON 형식**(음식 이름 · 영양소 · 확신도)으로만 받아요
3. 확신이 낮으면 **needs_review**로 표시하고, 서버에는 결과만 저장하고, 사진은 이 기기에만 보관해요
4. 함수는 **로그인한 사용자만** 부를 수 있고, 한 사람당 **하루 30회**까지만 받아요(횟수는 서버가 트랜잭션으로 세요)

왜 서버를 거치나: 키가 앱에 들어가면 앱 파일을 풀어서 누구나 꺼낼 수 있어요

B 친구 요청 — 규칙으로 권한 나누기

1. 초대 코드를 입력하면 **inviteCodes**에서 상대 uid를 찾아요
2. **friendRequests** 생성 — 규칙상 보낸 사람만 만들 수 있어요
3. 받은 사람이 수락(**받은 사람만 update**)하면 **friendships**가 생겨요
4. 활동 피드에는 짧은 한 줄만 남기고, 보이는 범위는 항목별로 사용자가 정해요

초대 코드는 헷갈리는 글자(0/O, 1/I)를 뺀 6자리예요

C 계정 삭제 — 순서가 중요한 흐름

1. **재인증**(민감 작업은 최근 로그인 필요) → Apple이면 **토큰 철회**
2. **내 데이터 삭제** — 기록 · 프로필 · 친구 · 챌린지 · 활동 피드(**300개씩 배치**)
3. 마지막에 **계정 삭제**

계정을 먼저 지우면 보안 규칙이 삭제를 허용하지 않아서, 데이터를 먼저 지워요

트러블슈팅

App Store 반려, 계정 삭제 뒤 남는 데이터, 기록 삭제 버그, 빌드 · 배포 문제 등 **막혔던 일 10가지**를 문제 → 원인 → 해결 순서로 따로 정리했어요.

cr-kstar.github.io/diet_project/troubleshooting/

권채린

Flutter 앱 개발 지망 · 신입

kchaerin0811@gmail.com github.com/CR-KStar/diet_project App Store 웹 앱 웹 체험판 개인정보 처리방침

※ iOS는 2026년 10월 16일 App Store에 출시했고, 웹 앱은 Firebase Hosting으로 배포했고, Android는 Google Play 출시 심사를 진행 중이에요(2026년 10월 기준).